



Creacode SIP Application Server[®] Configuration and Administration Guide

Revision History

Version	Date	Author	Remarks
1.0.7.0	18.08.2005	Arda Tekin Bahri Azar	First Draft
1.1.2.4	21.08.2009	Arda Tekin	Late documentation for redundant server installation and some configuration parameters of Registrar module
1.1.2.5	15.01.2010	Arda Tekin	Enhancement on Redundancy section and explaining KillOnlyProdProcess configuration
1.1.2.7	21.06.2013	Arda Tekin	CreacodeSAS rebranding changes
1.3.0.0	2025-12-12	Arda Tekin	Speech Services (STT / TTS) section added

Table of Contents

Application Directory	4
Configuration File	4
Installing CreacodeSAS Service.....	7
Call Control Scripts Compilation	7
Radius Dictionary Files	7
Speech Services (STT / TTS)	8
Proxy Services and Ports.....	8
Configuration and Credentials	8
Audio and Codec Compatibility	8
Language Handling	8
Starting the Speech Services	8
CreacodeSAS Telnet Commands.....	10
Executable Commands	10
Informational Commands	11
Redundancy.....	13

Application Directory

Directory Hierarchy starting form application path:

```

/APP_PATH
  /Bin
  /LOG
  /RadiusDictionary
  /Record
  /Scripts
  /Voice
    /turkish
      /G711U, /G711A, /G729A
    /english
      /G711U, /G711A, /G729A
    /german
      /G711U, /G711A, /G729A
    /french
      /G711U, /G711A, /G729A
    /russian
      /G711U, /G711A, /G729A

```

Bin	: NICIVR.exe, depended dll files and configuration file named ivr.ini, license file named NicIVRLic.lic are located here.
LOG	: Incules logging files named ivr.log, sip.log, rtp.log, script.log, radius.log, cdr.log.
RadiusDictionary	: Incudes RADIUS dictionary files which contains Vendor Specific Attributes(VSA) for specific vendors.
Record	: Recorded voice files for VoiceMail feature.
Scripts	: Script files to be used for call control.
Voice	: Multi-language voice files which are used when making announce.

Configuration File

CreacodeSAS configuration file **ivi.ini** is located in the same directory with application binary. Comment lines start with # character. Sections and parameter list:

[TCP]

Port	: CreacodeSAS listen port for SIP requests
Proxy	: SIP proxy IP
ProxyPort	: SIP proxy port
AutoDetectIP	: If 1, detects local IP automatically. 0 requires to define LocalIP.
LocalIP	: Machine IP
ReceiveBuffer	: UDP receive buffer. Default is 1024KB preferred 131072KB.
SendBuffer	: UDP send buffer. Default is 256KB.
RedundantListenIP	: Redundant server listen IP. Used when CreacodeSAS is in redundant mode.
RedundantListenPort	: Redundant server listen port. Default is 4000.
WatchDogIP	: Watchdog service listen IP
WatchDogPort	: Watchdog service listen port. Default is 4000.

[IVR].

VoiceLanguage	: Default voice language to be announced.
ScriptPath	: Call control scripts path.
ScriptPoolSize	: Number of script object to loaded by default. Optimal is license count.
VoiceDir	: Directory path for voice files.
RecordDir	: Recorded voice files path.
LogFile	: IVR log files path.
LogLevel	: Log level. Options: ALL, TRACE, DEBUG, INFO, WARN, ERROR, FATAL, OFF. Default is ERROR.
LogMaxFileSize	: Maximum log file size. File is truncated when the limit exceed. Default:5MB, -1=2GB.
LogMaxBackupCount	: Maximum log file count to be created. Default is 10.
TerminationDigit	: Interrupt digit in GetDigit() period (10=*, 11=#)
UserName	: CreacodeSAS username is used in register requests, re-invites or callbacks.

Register	: Registration option. 1 means to register which is defined in [TCP]/Proxy parameter. 0 means not register.
RegisterFreq	: Registration frequency in seconds.
RegisterFreqIfFail	: Time period between failed registration requests.
RegisterAuthUserName	: Authentication username used in REGISTER packet.
RegisterAuthPassword	: Authentication password used in REGISTER packet.
RegisterRealm	: Realm value used in REGISTER packet.
ProxyAuth	: Enables proxy authentication so that ivr can authenticate incoming INVITES. 1 means enabled, 0 means disabled.
[RTP]	
G711UAQuality	: G711U voice codec quality. Default is 3.
LogFile	: RTP log file path.
LogLevel	: Log level. Options: ALL, TRACE, DEBUG, INFO, WARN, ERROR, FATAL, OFF. Default is ERROR.
LogMaxFileSize	: Maximum log file size. File is truncated when the limit exceed. Default:5MB, -1=2GB.
LogMaxBackupCount	: Maximum log file count to be created. Default is 10.
ListenStartPort	: RTP listen start port number
ListenEndPort	: RTP listen end port number
DigitWaitTimeout	: Timeout, in seconds, between two digit when using GetDigit() function.
STTEngineType	: Speech-to-Text engine selection. Supported values: google-stt This parameter defines which STT backend is used by the STT() script function.
TTEngineType	: Text-to-Speech engine selection. Supported values: google-tts This parameter defines which TTS backend is used by the TTS() script function.
GCloudKeyPath	: Path to Google service account key file (gcloud_key.json). The file must contain valid Google Speech API credentials.
[SIP]	
LogFile	: SIP log file path.
LogLevel	: Log level. Options: ALL, TRACE, DEBUG, INFO, WARN, ERROR, FATAL, OFF. Default is ERROR.
LogMaxFileSize	: Maximum log file size. File is truncated when the limit exceed. Default:5MB, -1=2GB.
LogMaxBackupCount	: Maximum log file count to be created. Default is 10.
CallIdForwardingEnabled	: Forwards callid of LEG_A to LEG_B without changing it.
[Script]	
LogFile	: Scripting log file path.
LogMaxFileSize	: Maximum log file size. File is truncated when the limit exceed. Default:5MB, -1=2GB.
LogMaxBackupCount	: Maximum log file count to be created. Default is 10.
[Radius]	
Secret	: Secret for radius encryption.
DictionaryPath	: RADIUS VSA dictionary path.
LogFile	: SIP log file path.
LogLevel	: Log level. Options: ALL, TRACE, DEBUG, INFO, WARN, ERROR, FATAL, OFF. Default is ERROR.
LogMaxFileSize	: Maximum log file size. File is truncated when the limit exceed. Default:5MB, -1=2GB.
LogMaxBackupCount	: Maximum log file count to be created. Default is 10.
PacketRetryCount	: Radius packet send retry count. Default is 0.
PacketTimeout	: Radius packet send timeout. Default is 5 seconds.
[VoiceMail]	
MailServer	: Mail server when sending recorded voice files.
From	: VoiceMail From information
Subject	: VoiceMail Subject information
[Telnet]	
UserName	: Telnet username. Default is nicivr.
Password	: Encrypted telnet password. Decrypt on runtime. Default is nicivr. Encrypted is 345f5954444742.
Port	: Telnet port
[Database]	
DSN	: ODBC Data Source Name



User : Database user name
Password : Encrypted DSN password. Decrypt on runtime.
Default is nicivr. Encrypted is 345f5954444742.

[CDR]

LogFile : CDR log file path.
LogLevel : Log level.
Options: ALL, TRACE, DEBUG, INFO, WARN, ERROR, FATAL, OFF. Default is ERROR.
LogMaxFileSize : Maximum log file size. File is truncated when the limit exceed.
Default:5MB, -1=2GB.
LogMaxBackupCount : Maximum log file count to be created. Default is 10.
Log Pattern : Indicates CDR log format. Supported keywords: Now; SetupTime; ConnectTime;
DisconnectTime; CPN; DNIS; Route; RoutePort; SessionTime; DisconnectCouse;
ClosingLeg

[REGISTRAR]

Enabled : 0 Disabled, 1 enabled
Realm : SIP authentication parameter
DefaultExpire : Used when client doesn't send expire header
ExpireCheckInterval : Internal parameter. It must be set 10 by default.



Installing CreacodeSAS Service

To install the CreacodeSAS service run APP_PATH/Bin/service_install.bat file.

To remove the service run APP_PATH/Bin/service_remove.bat file.

Call Control Scripts Compilation

SCC.exe binary is used for CreacodeSAS script compilation. Compiler gets 2 parameters:

SCC.exe [Startup Script File] out.sc

Compiler is generally put into script directory. Output named out.sc is created when compilation is successful.

out.sc includes call control information which will be used when processing incoming call.

Radius Dictionary Files

All Radius standart attributes are defined by default in CreacodeSAS. APP_PATH/RadiusDictionary directory includes Radius Vendor Specific Attribute(VSA) definition files. VSA definition files are in the following format:

VENDOR	[vendor_name]	[vendor_id]		
ATTRIBUTE	[attribute_name]	[attribute_id]	[attribute_type]	[vendor_name]

Each dictionary files are read by CreacodeSAS when starting up.

Speech Services (STT / TTS)

CreacodeSAS provides Speech-to-Text (STT) and Text-to-Speech (TTS) capabilities through external Python-based proxy services. These services enable real-time voice interaction by converting RTP audio streams into text and synthesizing speech dynamically during call execution. Speech services are intentionally implemented outside the core C++ runtime. This decoupled design allows flexible backend selection, independent lifecycle management, and future extensibility without requiring changes or recompilation of the main IVR engine.

During a call, CreacodeSAS captures RTP audio from the active call leg and forwards it to the corresponding speech proxy service. The proxy communicates with the configured cloud speech engine and returns either recognized text (STT) or synthesized audio (TTS) back to the IVR runtime, where it is processed synchronously within the script flow.

Both STT and TTS services operate in parallel with the IVR runtime and must be running and reachable before the related script functions (STT() or TTS()) are invoked.

Proxy Services and Ports

Speech processing is handled by the following Python scripts:

- STTProxy.py — Speech-to-Text proxy (default listen port: 8089)
- TTSPProxy.py — Text-to-Speech proxy (default listen port: 8090)

These scripts listen on predefined TCP ports for requests coming from CreacodeSAS. The listening ports must be available on the host system and **must not be in use by any other application**. If a port conflict exists, speech services will fail to initialize or accept requests.

Both proxy scripts reside **next to CreacodeSAS.exe under the Bin directory**, ensuring a consistent runtime layout and simplified deployment.

Configuration and Credentials

Speech services are configured through the ivr.ini file. Engine selection and credential location are defined using the following parameters:

- STTEngineType selects the Speech-to-Text backend.
- TTSEngineType selects the Text-to-Speech backend.
- GCloudKeyPath defines the location of the Google service account key file.

When using Google Speech services, **Google Speech-to-Text and Google Text-to-Speech APIs must be enabled in the Google Cloud Console**. A valid service account key must be generated and stored in a file named gcloud_key.json.

The gcloud_key.json file must be accessible at runtime and is expected to reside next to the CreacodeSAS.exe binary, under the Bin deployment directory.

Audio and Codec Compatibility

Speech services operate on RTP audio captured directly from the call. The current implementation assumes standard telephony parameters, including G.711 (μ -law or A-law) encoding, 8 kHz sample rate, and mono audio. These settings are compatible with Google Speech APIs. Any required decoding or preprocessing is handled within the proxy layer.

Language Handling

The active script language, defined by the `_LANGUAGE_` variable in the IVR scripting environment, determines both the STT recognition language and the TTS voice and synthesis language. This allows multilingual call flows and dynamic language switching at runtime without changing the underlying service configuration.

Starting the Speech Services

At present, the speech proxy services can be started manually. In future releases, these services will be automatically started and managed by the deployment packages.

To start the services manually from PowerShell:



```
cd Bin
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\sttvenv\Scripts\Activate.ps1
python .\STTProxy.py
python .\TTSProxy.py
```

Once started, both proxy services must remain running for speech recognition and synthesis to function correctly during calls.



CreacodeSAS Telnet Commands

Executable Commands

compile [first script file]

first script file : Starts to compile scripts from this file. Default is "Main.txt".

Generates out.sc file which will be used for session call flow process into the script path.

crypt [crypt string]

crypt string : string to be crypted

Crypts a string and generates crypted output.

decrypt [decrypt string]

decrypt string : string to be decrypted

Decrypts a string and generates decrypted output.

route [print]
 [add order_id=FV [cpn=FV] [dnis=FV] [source_ip_start=FV] [source_ip_end=FV]
 [rule=FV] [proxy_ip=FV] [proxy_port=FV] [domain_name=FV]]
 [delete order_id=FV]
 [update order_id=FV [cpn=FV] [dnis=FV] [source_ip_start=FV] [source_ip_end=FV]
 [rule=FV] [proxy_ip=FV] [proxy_port=FV] [domain_name=FV]]
 [reload]

print : Prints all rules in order.
 add : Adds a route and reloads route table.
 delete : Deletes route for specified route id and reloads route table.
 update : Updates an existing route and reloads route table. order_id is required and other parameters are optional.
 reload : Reloads route table.
 order_id : route unique id. Required.
 cpn : calling station number
 dnis : called station number
 source_ip_start : Start IP for incoming call IP range
 source_ip_end : End IP for incoming call IP range
 rule : Rules to be applied to destination number. Processed in order.
 Example: 0212=90212;0216=90216
 proxy_ip : SIP proxy IP
 proxy_port : SIP proxy port. If not define, default is 5060.
 domain_name : domain

Route plan command. FV is field value for each database field.

Example:

```
route add order_id=2 cpn=90* dnis=90212* source_ip_start=212.56.35.1
source_ip_end=212.56.35.255 rule=90=0; proxy_ip=212.56.36.32
proxy_port=5060 domain_name=foo.com
```

```
route update order_id=2 proxy_ip=212.56.36.35 proxy_port=5050
```

acl [print]
 [add order_id=FV source_ip_start=FV source_ip_end=FV allowed=FV]
 [delete order_id=FV]
 [update order_id=FV source_ip_start=FV source_ip_end=FV allowed=FV]
 [reload]

print : Prints ACL table in order.
 add : Adds an ACL entry and reloads ACL table.
 delete : Deletes for specified ACL id and reloads ACL table.



update	: Updates an existing ACL entry and reloads ACL table. order_id is required and other parameters are optional.
reload	: Reloads ACL table.
order_id	: ACL unique id. Required.
source_ip_start	: Start IP for incoming call IP range. Required.
source_ip_end	: End IP for incoming call IP range. Required.
allowed	: 1 allows, 0 denies. Required.

Access control list command. FV is field value for each database field.

Example:

```
acl add order_id=5 source_ip_start=212.56.35.1 source_ip_end=212.56.35.255 allowed=0
acl update order_id=5 allowed=1
```

kill [sessionid [notbill] [direct]]

sessionid	: unique session id in the current system. ("list session" command report running sessions)
notbill	: kills the session without billing the call duration.
direct	: kills the session only for SIP signalling.

Terminates the specified session.

"kill 'sessionid' " command processes following order to kill the session.

- Sends SIP BYE message to LEG_A
- Sets CLOSING_LEG = LEG_A
- Calls CallEnd() Event in the current script
- GetDuration() function returns real session duration
- Calls ReleaseSession() function
- Sending SIP BYE message to LEG_B is the responsibility of scripting developer

"kill 'notbill' " command processes following order to kill the session.

- Sends SIP BYE message to LEG_A
- Sets CLOSING_LEG = LEG_A
- Calls CallEnd() Event in the current script
- GetDuration() function returns zero duration
- Calls ReleaseSession() function
- Sending SIP BYE message to LEG_B is the responsibility of scripting developer

"kill 'direct' " command processes following order to kill the session.

- Sends SIP BYE message to LEG_A and LEG_B
- Calls ReleaseSession() function

reload [voice]

voice	: Reloads all-voice files.
-------	----------------------------

log [sip] [ivr] [rtp] [script] [radius]

sip	: prints SIP log
ivr	: prints internal ivr log
rtp	: prints RTP channel log
script	: prints scripting log
radius	: prints radius log

Prints specified log to telnet window. "q" command interrupts the process.

exit

Exits telnet session.

Informational Commands

list [session]



session : lists all running sessions. Reports Unique SessionID, CPN, DNIS and Call Receive Time for each session.

Lists the specified unit.

count	[Session] [RTPChannel] [TotalCall] [SuccessCall] [FailCall]
Session	: Active session controller count
RTPChannel	: Active(open) RTP channel count
TotalCall	: Total incoming call count
SuccessCall	: Successful call count. Increased 1 when incoming call is connected to destination over CreacodeSAS.
FailCall	: Failed call count. Increase 1 when incoming call could not connect to destination.

Count of specified unit.

peak

Reports the peak and peak time for incoming calls(sessions).

help

Prints telnet commands.

Redundancy

The role of CreacodeSAS Redundant server is to periodically check the availability of main server and take over the operation in case of main server is down. To do that, redundant server communicates to main server over WatchDog service. Please note that redundant server can not be run before main server's state become down.

The relation between Main Server(MS) – Watchdog Service(WD) – Redundant Server(RS) is as below;

- RS starts to run on separate machine and sends "control request" to WD service, and waits for OK response from WD by listening RedundantIP:RedundantPort.
- When WD gets "control request", it sends SIP OPTIONS packet to MS and waits answer from MS. If MS doesn't answer with 200OK in a certain period of time then WD shuts down the ethernet interface of MS and sends MS's IP address to RS.
- RS adds this IP address onto its own network adapter and starts sip engine using that IP address.
- Then incoming SIP messages are redirected to RS by the routers and gateways in the network.

Please follow the instructions below to configure each node correctly;

- I. MS and WD must run on the same machine and listen on separate ethernet adapters. MS and RS must be located in the same subnet.
- II. Copy NICWD.exe, wd_install.bat, wd_remove.bat and wd.ini into "bin" directory.
- III. Run wd_install.bat and install WD service on MS machine.
- IV. Install RS on another machine which places in the same subnet and runs as redundant. RS installation requires RS software license. Please ask RS license to your vendor.
- V. Configure wd.ini of WD as below;
 - Set logfile path of WD
 - Set WD ListenIP:ListenPort so that WD can receive "control request" from RS and status responses from MS.
 - Set ProdIP and ProdPort(always 5060) to connect WD to MS to get the availability status of MS.
 - Set RedundantIP:RedundantPort to connect WD to RS. So WD could deliver IP address of MS to RS.
 - Set KillOnlyProdProcess as 0.Then run WD service.
- VI. Configure ivr.ini of RS as below;
 - Set WatchDogIP and WatchDogPort to send the "control request" to WD.
 - Set RedundantListenIP and RedundantListenPort, so that RS can receive "alive" result from WD service.
 - The rest of ivr.ini is same as ivr.ini of MSRun RS service.

If you set KillOnlyProdProcess as 1, WD ignores the "control request" of RS and sends SIP OPTIONS to MS itself to check that MS is alive. If WD doesn't get 200OK answer in the specified period of time and after retries, WD directly kills NICIVR.exe process and stops its execution.